CS210 Assignment 6

Author: Matthew Woo

Date: Oct 18, 2026

AI Disclosure: I used ChatGPT 6 to help me to get started. I also used AI to learn how to implement each collision mitigation strategy and how each works.

Source Code

```
/** AI Disclosure: I used ChatGPT 6 to help me to get started.
 *  I also used AI to learn how to implement each collision mitigation strategy and how each works.
*/
#include <iostream>
#include <string>
#include <vector>
#include <cstdint>
using namespace std;

struct User {
    string username;
    string salt;
    uint64_t passwordHash;
};

struct Node {
    User data;
    Node* next;
};

uint64_t demoHash(const std::string& s) {
    uint64_t h = 14695981039346656037ULL;
    for (unsigned char c : s) {
        h = (h ^ c) * 1099511628211ULL;
    }
    return h;
}

const string pepper = "pwdPepper";

//Chaining
class HashTable {
private:
    vector<Node*> table;
    int userCount;

    int hashFunction(string username) {
        int sum = 0;
        for (char c : username) {
            sum += c;
        }
        return sum % table.size(); //because we have 5 buckets
    }

    void rehash() {
        int oldSize = table.size();
        vector<Node*> oldTable = table;
        // Create a bigger table
        table.clear();
        table.resize(oldSize * 2 + 1, nullptr);
        // Move users to the new table
        for (int i = 0; i < oldSize; i++) {
            Node* current = oldTable[i];
            while (current != nullptr) {
                Node* nextNode = current->next;
                int index = hashFunction(current->data.username);
                current->next = table[index];
                table[index] = current;
                current = nextNode;
            }
        }
        cout << "Table resized from " << oldSize
            << " to " << table.size() << endl;
    }
```

```
public:
    HashTable() {
        table.resize(5, nullptr);
        userCount = 0;
    }

    void insert(User user) {
        if (find(user.username) != nullptr) {
            cout << "Username already exists!" << endl;
            return;
        }
        int index = hashFunction(user.username);
        Node* newNode = new Node;
        newNode->data = user;
        newNode->next = table[index];
        table[index] = newNode;
        userCount++;
        cout << user.username << " inserted at bucket "
            << index << endl;
        if (loadFactor() > 0.75) {
            rehash();
        }
    }

    User* find(string username) {
        int index = hashFunction(username);
        Node* current = table[index];
        while (current != nullptr) {
            if (current->data.username == username) {
                return &current->data;
            }
            current = current->next;
        }
        return nullptr;
    }

    bool remove(string username) {
        int index = hashFunction(username);
        Node* current = table[index];
        Node* previous = nullptr;
        while (current != nullptr) {
            if (current->data.username == username) {
                if (previous == nullptr) {
                    table[index] = current->next;
                }
                else {
                    previous->next = current->next;
                }
                delete current;
                userCount--;
                return true;
            }
            previous = current;
            current = current->next;
        }
        return false;
    }

    double loadFactor() {
        return (double)userCount / table.size();
    }
};

//Linear Hash
class LinearHashTable {
private:
    vector<User> table;
    vector<bool> occupied;
    vector<bool> deleted;
    int userCount = 0;
    int hashFunction(string username) {
        int sum = 0;
        for (char c : username) {
            sum += c;
        }
        return sum % table.size();
    }

    void rehash() {
        int oldSize = table.size();
        vector<User> oldTable = table;
        vector<bool> oldOccupied = occupied;
```

```
        // Create a bigger table
        table.clear();
        table.resize(oldSize * 2 + 1);
        occupied.assign(table.size(), false);
        deleted.assign(table.size(), false);
        // Move users to the new table
        for (int i = 0; i < oldSize; i++) {
            if (oldOccupied[i]) {
                int index = hashFunction(oldTable[i].username);
                while (occupied[index]) {
                    index = (index + 1) % table.size();
                }
                table[index] = oldTable[i];
                occupied[index] = true;
            }
        }
        cout << "Table resized from " << oldSize
            << " to " << table.size() << endl;
    }

public:
    LinearHashTable() {
        table.resize(5);
        occupied.resize(5, false);
        deleted.resize(5, false);
    }
    void insert(User user) {
        if (find(user.username) != nullptr) {
            cout << "Username already exists!" << endl;
            return;
        }
        int index = hashFunction(user.username);
        for (int i = 0; i < table.size(); i++) {
            //If it is not occupied then store it and mark it as occupied
            if (!occupied[index]) {
                table[index] = user;
                occupied[index] = true;
                userCount++;
                deleted[index] = false;
                cout << user.username << " inserted at bucket "
                    << index << endl;
                if (loadFactor() > 0.75) {
                    rehash();
                }
                return;
            }
            //If occupied, check the next empty bucket
            index = (index + 1) % table.size();
        }
        cout << "Hash table is full!" << endl;
    }

    User* find(string username) {
        int index = hashFunction(username);
        for (int i = 0; i < table.size(); i++) {
            if (!occupied[index] && !deleted[index]) {
                return nullptr;
            }
            if (occupied[index] && table[index].username == username) {
                return &table[index];
            }
            index = (index + 1) % table.size();
        }
        return nullptr;
    }

    bool remove(string username) {
        int index = hashFunction(username);
        for (int i = 0; i < table.size(); i++) {
            if (!occupied[index] && !deleted[index]) {
                return false;
            }
            if (occupied[index] && table[index].username == username) {
                occupied[index] = false;
                deleted[index] = true;
                userCount--;
                return true;
            }
            index = (index + 1) % table.size();
        }
        return false;
    }
```

```
    double loadFactor() {
        return (double)userCount / table.size();
    }
};

//Quadratic Hash
class QuadraticHashTable {
private:
    vector<User> table;
    vector<bool> occupied;
    vector<bool> deleted;
    int userCount = 0;
    int hashFunction(string username) {
        int sum = 0;
        for (char c : username) {
            sum += c;
        }
        return sum % table.size();
    }

    bool isPrime(int n) {
        if (n < 2) {
            return false;
        }

        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) {
                return false;
            }
        }

        return true;
    }


    void rehash() {
        int oldSize = table.size();
        vector<User> oldTable = table;
        vector<bool> oldOccupied = occupied;
        // Find a larger prime table size
        int newSize = oldSize * 2 + 1;
        while (!isPrime(newSize)) {
            newSize++;
        }
        // Create a bigger table
        table.clear();
        table.resize(newSize);
        occupied.assign(newSize, false);
        deleted.assign(newSize, false);
        // Move users to the new table
        for (int i = 0; i < oldSize; i++) {
            if (oldOccupied[i]) {
                int startIndex = hashFunction(oldTable[i].username);
                for (int j = 0; j < newSize; j++) {
                    int index = (startIndex + j * j) % newSize;
                    if (!occupied[index]) {
                        table[index] = oldTable[i];
                        occupied[index] = true;
                        break;
                    }
                }
            }
        }
        cout << "Table resized from " << oldSize
            << " to " << newSize << endl;
    }

public:
    QuadraticHashTable() {
        table.resize(5);
        occupied.resize(5, false);
        deleted.resize(5, false);
    }
    void insert(User user) {
        if (find(user.username) != nullptr) {
            cout << "Username already exists!" << endl;
            return;
        }
        // Resize before inserting if load factor would exceed 0.5
        if ((double)(userCount + 1) / table.size() > 0.5) {
            rehash();
```

```
        }
        int startIndex = hashFunction(user.username);
        for (int i = 0; i < table.size(); i++) {
            int index = (startIndex + i * i) % table.size();
            if (!occupied[index]) {
                table[index] = user;
                occupied[index] = true;
                deleted[index] = false;
                userCount++;
                cout << user.username << " inserted at bucket "
                     << index << endl;
                return;
            }
        }
        cout << "Could not insert user!" << endl;
    }

    User* find(string username) {
        int startIndex = hashFunction(username);
        for (int i = 0; i < table.size(); i++) {
            int index = (startIndex + i * i) % table.size();
            if (!occupied[index] && !deleted[index]) {
                return nullptr;
            }
            if (occupied[index] && table[index].username == username) {
                return &table[index];
            }
        }
        return nullptr;
    }

    bool remove(string username) {
        int startIndex = hashFunction(username);
        for (int i = 0; i < table.size(); i++) {
            int index = (startIndex + i * i) % table.size();
            if (!occupied[index] && !deleted[index]) {
                return false;
            }
            if (occupied[index] && table[index].username == username) {
                occupied[index] = false;
                deleted[index] = true;
                userCount--;
                return true;
            }
        }
        return false;
    }

    double loadFactor() {
        return (double)userCount / table.size();
    }


};

//Double Hashing
class DoubleHashTable {
private:
    vector<User> table;
    vector<bool> occupied;
    vector<bool> deleted;
    int userCount = 0;
    int hashFunction(string username) {
        int sum = 0;
        for (char c : username) {
            sum += c;
        }
        return sum % table.size();
    }
    int secondHash(string username) {
        int sum = 0;
        for (char c : username) {
            sum += c;
        }
        return 1 + (sum % (table.size() - 1));
    }

    bool isPrime(int n) {
        if (n < 2) {
            return false;
        }
        for (int i = 2; i * i <= n; i++) {
```

```
            if (n % i == 0) {
                return false;
            }
        }
        return true;
    }

    void rehash() {
        int oldSize = table.size();
        vector<User> oldTable = table;
        vector<bool> oldOccupied = occupied;
        // Find a larger prime table size
        int newSize = oldSize * 2 + 1;
        while (!isPrime(newSize)) {
            newSize++;
        }
        // Create a bigger table
        table.clear();
        table.resize(newSize);
        occupied.assign(newSize, false);
        deleted.assign(newSize, false);
        // Move users to the new table
        for (int i = 0; i < oldSize; i++) {
            if (oldOccupied[i]) {
                int startIndex = hashFunction(oldTable[i].username);
                int step = secondHash(oldTable[i].username);
                for (int j = 0; j < newSize; j++) {
                    int index = (startIndex + j * step) % newSize;
                    if (!occupied[index]) {
                        table[index] = oldTable[i];
                        occupied[index] = true;
                        break;
                    }
                }
            }
        }
        cout << "Table resized from " << oldSize
            << " to " << newSize << endl;
    }

public:
    DoubleHashTable() {
        table.resize(5);
        occupied.resize(5, false);
        deleted.resize(5, false);
    }
    void insert(User user) {
        if (find(user.username) != nullptr) {
            cout << "Username already exists!" << endl;
            return;
        }
        int startIndex = hashFunction(user.username);
        int step = secondHash(user.username);
        for (int i = 0; i < table.size(); i++) {
            int index = (startIndex + i * step) % table.size();
            if (!occupied[index]) {
                table[index] = user;
                occupied[index] = true;
                deleted[index] = false;
                userCount++;
                cout << user.username << " inserted at bucket "
                    << index << endl;
                if (loadFactor() > 0.75) {
                    rehash();
                }
                return;
            }
        }
        cout << "Hash table is full!" << endl;
    }

    User* find(string username) {
        int startIndex = hashFunction(username);
        int step = secondHash(username);
        for (int i = 0; i < table.size(); i++) {
            int index = (startIndex + i * step) % table.size();
            if (!occupied[index] && !deleted[index]) {
                return nullptr;
            }
            if (occupied[index] && table[index].username == username) {
                return &table[index];
            }
```

```
        }
        return nullptr;
    }

    bool remove(string username) {
        int startIndex = hashFunction(username);
        int step = secondHash(username);
        for (int i = 0; i < table.size(); i++) {
            int index = (startIndex + i * step) % table.size();
            if (!occupied[index] && !deleted[index]) {
                return false;
            }
            if (occupied[index] && table[index].username == username) {
                occupied[index] = false;
                deleted[index] = true;
                userCount--;
                return true;
            }
        }
        return false;
    }

    double loadFactor() {
        return (double)userCount / table.size();
    }
};

void login(HashTable& users) {
    string username;
    string password;
    cout << "Enter username: ";
    cin >> username;
    cout << "Enter password: ";
    cin >> password;
    User* user = users.find(username);
    if (user == nullptr) {
        cout << "Unknown user!" << endl;
    }
    else if (demoHash(user->salt + password + pepper) == user->passwordHash) {
        cout << "Login successful!" << endl;
    }
    else {
        cout << "Wrong password!" << endl;
    }
}

int main() {
    User alice = {"alice", "saltAlice", demoHash("saltAlice" + string("alice123") + pepper)};
    User bob = {"bob", "saltBob", demoHash("saltBob" + string("bob123") + pepper)};
    User carol = {"carol", "saltCarol", demoHash("saltCarol" + string("carol123") + pepper)};
    User david = {"david", "saltDavid", demoHash("saltDavid" + string("david123") + pepper)};
    User erin = {"erin", "saltErin", demoHash("saltErin" + string("erin123") + pepper)};
    HashTable users;
    users.insert(alice);
    users.insert(bob);
    users.insert(carol);
    users.insert(david);
    users.insert(erin);
    cout << "\nTotal load factor: " << users.loadFactor() << endl;
    // Test Rehash
    cout << "\nTesting Rehash:" << endl;
    cout << "Load factor: " << users.loadFactor() << endl;
    if (users.find("bob") != nullptr) {
        cout << "Bob found after rehash!" << endl;
    }
    else {
        cout << "Bob not found!" << endl;
    }
    // Login
    cout << "\n===== LOGIN SYSTEM =====" << endl;
    login(users);
    // Removal Test
    cout << "\nRemoving Alice..." << endl;
    if (users.remove("alice")) {
        cout << "Alice removed successfully!" << endl;
    }
    else {
        cout << "User not found!" << endl;
    }
    cout << "\nTry logging in again:" << endl;
    login(users);
```

```
    // Test Linear Probing
    cout << "\n===== LINEAR PROBING =====" << endl;
    LinearHashTable linear;
    linear.insert(alice);
    linear.insert(erin);
    linear.insert(bob);
    linear.insert(david);
    linear.insert(carol);
    cout << "Load factor: " << linear.loadFactor() << endl;
    cout << "Find Erin: " << (linear.find("erin") != nullptr) << endl;
    cout << "Remove Alice: " << linear.remove("alice") << endl;
    cout << "Erin still found: " << (linear.find("erin") != nullptr) << endl;
    cout << "Remove unknown user: " << linear.remove("john") << endl;

    // Test Quadratic Probing
    cout << "\n===== QUADRATIC PROBING =====" << endl;
    QuadraticHashTable quadratic;
    quadratic.insert(alice);
    quadratic.insert(erin);
    quadratic.insert(david);
    quadratic.insert(bob);
    quadratic.insert(carol);
    cout << "Load factor: " << quadratic.loadFactor() << endl;
    cout << "Find Bob: " << (quadratic.find("bob") != nullptr) << endl;
    cout << "Remove Alice: " << quadratic.remove("alice") << endl;
    cout << "Erin still found: " << (quadratic.find("erin") != nullptr) << endl;

    // Test Double Hashing
    cout << "\n===== DOUBLE HASHING =====" << endl;
    DoubleHashTable doubleHash;
    doubleHash.insert(alice);
    doubleHash.insert(erin);
    doubleHash.insert(bob);
    doubleHash.insert(david);
    doubleHash.insert(carol);
    cout << "Load factor: " << doubleHash.loadFactor() << endl;
    cout << "Find Erin: " << (doubleHash.find("erin") != nullptr) << endl;
    cout << "Remove Alice: " << doubleHash.remove("alice") << endl;
    cout << "David still found: " << (doubleHash.find("david") != nullptr) << endl;

    return 0;
}
```

Output/Test Cases

Successful login, unknown user, removal, and rehash

```
alice inserted at bucket 0
bob inserted at bucket 2
carol inserted at bucket 4
david inserted at bucket 0
Table resized from 5 to 11
erin inserted at bucket 1

Total load factor: 0.454545

Testing Rehash:
Load factor: 0.454545
Bob found after rehash!

===== LOGIN SYSTEM =====
Enter username: alice
Enter password: alice123
Login successful!

Removing Alice...
Alice removed successfully!

Try logging in again:
Enter username: alice
Enter password: alice123
Unknown user!

===== LINEAR PROBING =====
alice inserted at bucket 0
erin inserted at bucket 1
bob inserted at bucket 2
david inserted at bucket 3
```

```
Table resized from 5 to 11
carol inserted at bucket 2
Load factor: 0.454545
Find Erin: 1
Remove Alice: 1
Erin still found: 1
Remove unknown user: 0

===== QUADRATIC PROBING =====
alice inserted at bucket 0
erin inserted at bucket 1
Table resized from 5 to 11
david inserted at bucket 3
bob inserted at bucket 10
carol inserted at bucket 2
Load factor: 0.454545
Find Bob: 1
Remove Alice: 1
Erin still found: 1

===== DOUBLE HASHING =====
alice inserted at bucket 0
erin inserted at bucket 3
bob inserted at bucket 2
david inserted at bucket 1
Table resized from 5 to 11
carol inserted at bucket 0
Load factor: 0.454545
Find Erin: 1
Remove Alice: 1
David still found: 1
```

Failed password, unknown user, removal, and rehash

```
david inserted at bucket 3
Table resized from 5 to 11
carol inserted at bucket 2
Load factor: 0.454545
Find Erin: 1
Remove Alice: 1
Erin still found: 1
Remove unknown user: 0

===== QUADRATIC PROBING =====
alice inserted at bucket 0
erin inserted at bucket 1
Table resized from 5 to 11
david inserted at bucket 3
bob inserted at bucket 10
carol inserted at bucket 2
Load factor: 0.454545
Find Bob: 1
Remove Alice: 1
Erin still found: 1

===== DOUBLE HASHING =====
alice inserted at bucket 0
erin inserted at bucket 3
bob inserted at bucket 2
david inserted at bucket 1
Table resized from 5 to 11
carol inserted at bucket 0
Load factor: 0.454545
Find Erin: 1
Remove Alice: 1
David still found: 1
matthewoo@Matthews-MacBook-Pro Downloads % clear
matthewoo@Matthews-MacBook-Pro Downloads % /Users/matthewoo/Downloads/Woo_Matthew_Assignment6
alice inserted at bucket 0
bob inserted at bucket 2
carol inserted at bucket 4
david inserted at bucket 0
Table resized from 5 to 11
erin inserted at bucket 1

Total load factor: 0.454545

Testing Rehash:
Load factor: 0.454545
Bob found after rehash!
```

```
===== LOGIN SYSTEM =====
Enter username: david
Enter password: apple123
Wrong password!

Removing Alice...
Alice removed successfully!

Try logging in again:
Enter username: alice
Enter password: alice123
Unknown user!

===== LINEAR PROBING =====
alice inserted at bucket 0
erin inserted at bucket 1
bob inserted at bucket 2
david inserted at bucket 3
Table resized from 5 to 11
carol inserted at bucket 2
Load factor: 0.454545
Find Erin: 1
Remove Alice: 1
Erin still found: 1
Remove unknown user: 0

===== QUADRATIC PROBING =====
alice inserted at bucket 0
erin inserted at bucket 1
Table resized from 5 to 11
david inserted at bucket 3
bob inserted at bucket 10
carol inserted at bucket 2
Load factor: 0.454545
Find Bob: 1
Remove Alice: 1
Erin still found: 1

===== DOUBLE HASHING =====
alice inserted at bucket 0
erin inserted at bucket 3
bob inserted at bucket 2
david inserted at bucket 1
Table resized from 5 to 11
carol inserted at bucket 0
Load factor: 0.454545
Find Erin: 1
Remove Alice: 1
David still found: 1
```

Choice of the best mitigation strategy after implementing all of them

I chose chaining because it is easy to implement using linked lists. It is also easier to insert and remove users compared to other methods.

Explain collisions, chaining, load factor, expected lookup complexity, and why a classroom hash is not a production password-hashing scheme.

Collisions: When two users produce the same index, it is a collision.

Chaining: It stores two users in the same index when a collision happens.

Load Factor: Load factor is the number of users divided by the number of buckets.

Expected Lookup Complexity: The expected lookup time is O(1) because it lookup the user index and goes there directly. In the worst case, it is O(n) when many users have the same index in the linked list.

Password Security: The demoHash() is not for real password security because it is too fast and it is easy guessing for some computer. We should use secure password-hashing algorithms like Argon2 for real password security.